

ЛАБОРАТОРНАЯ РАБОТА №6

«ИССЛЕДОВАНИЕ АЛГОРИТМОВ ПРОГРАММИРОВАНИЯ С ИСПОЛЬЗОВАНИЕМ МОДУЛЕЙ»

1. Цель работы

Приобретение навыков разработки и отладки программ, использующих модули.

2. Постановка задачи и вариант задания

Программу из лабораторной работы №5 или программу КП необходимо оформить в виде модулей.

ХОД РАБОТЫ

3. Текст кода из файла main.c

```
#include "tree.h"
#include "ui.h"
#include <windows.h>

int main() {
    SetConsoleCP(65001);
    SetConsoleOutputCP(65001);

    struct tree *top = NULL;
    runProgram(&top);

    return 0;
}
```

4. Текст кода из файла tree.h

```
#ifndef TREE_H
#define TREE_H

#include <stdio.h>

// Структура данных учёного
struct scientist {
    int id;
    char name[76];
    char area[26];
    char degree[26];
    int articles;
    int quotes;
    int hirshIndex;
};

// Узел бинарного дерева (ключ - количество статей)
struct tree {
    struct scientist info;
```

```

        struct tree *left, *right;
    };

// Функции для работы с деревом
struct tree *addNode(struct tree *top, struct scientist data);
struct tree *deleteNode(struct tree *node, int articles);
void viewTree(struct tree *top);
void printTreeStructure(struct tree *node, int space, int indent);
void saveToFile(struct tree *node, FILE *f);
struct tree *loadFromFile(FILE *f);
int findRightmostLevel(struct tree *node);
void printScientistsWithMoreThan100Articles(struct tree *node, int *count);
void freeTree(struct tree *node);

// Вспомогательная для удаления
struct tree *findMin(struct tree *node);

#endif

```

5. Текст кода из файла tree.c

```
#include "tree.h"
#include <stdlib.h>
#include <string.h>

// Статическая функция вывода одного узла (используется внутри модуля)
static void printNode(struct tree *node) {
    printf("| %-4d | %-75s | %-25s | %-25s | %-13d | %-18d | %-12d |\n",
        node->info.id,
        node->info.name,
        node->info.degree,
        node->info.area,
        node->info.articles,
        node->info.quotes,
        node->info.hirshIndex);
    printf("+-----+-----+-----+-----+-----+-----+\n");
}

// Добавление узла (по ключу articles)
struct tree *addNode(struct tree *top, struct scientist data) {
    if (!top) {
        top = (struct tree*)malloc(sizeof(struct tree));
        if (!top) {
            printf("Ошибка выделения памяти!\n");
            return NULL;
        }
        top->info = data;
        top->left = top->right = NULL;
    } else if (data.articles <= top->info.articles) {
        top->left = addNode(top->left, data);
    } else {
        top->right = addNode(top->right, data);
    }
    return top;
}

// Поиск минимального узла (для удаления)
struct tree *findMin(struct tree *node) {
    while (node && node->left) node = node->left;
    return node;
}
```

```

// Удаление узла по количеству статей
struct tree *deleteNode(struct tree *node, int articles) {
    if (node == NULL) {
        printf("Учёный с количеством статей %d не найден.\n", articles);
        return NULL;
    }
    if (articles < node->info.articles) {
        node->left = deleteNode(node->left, articles);
    } else if (articles > node->info.articles) {
        node->right = deleteNode(node->right, articles);
    } else {
        if (node->left == NULL) {
            struct tree *temp = node->right;
            free(node);
            return temp;
        } else if (node->right == NULL) {
            struct tree *temp = node->left;
            free(node);
            return temp;
        }
        struct tree *minNode = findMin(node->right);
        node->info = minNode->info;
        node->right = deleteNode(node->right, minNode->info.articles);
    }
    return node;
}

// Симметричный обход (вывод в таблицу)
void viewTree(struct tree *top) {
    if (top) {
        viewTree(top->left);
        printNode(top);
        viewTree(top->right);
    }
}

// Вывод структуры дерева (повёрнуто налево)
void printTreeStructure(struct tree *node, int space, int indent) {
    if (node == NULL) return;
    space += indent;
    printTreeStructure(node->right, space, indent);
    printf("\n");
    for (int i = indent; i < space; i++) printf(" ");
    printf("%d (%s) [статей: %d]\n", node->info.id, node->info.name, node->info.articles);
    printTreeStructure(node->left, space, indent);
}

// Сохранение дерева в файл (pre-order с маркерами 'D' и 'N')
void saveToFile(struct tree *node, FILE *f) {
    if (node == NULL) {
        fprintf(f, "N\n");
        return;
    }
    fprintf(f, "D\n");
    fprintf(f, "%d\n", node->info.id);
    fprintf(f, "%s\n", node->info.name);
    fprintf(f, "%s\n", node->info.area);
    fprintf(f, "%s\n", node->info.degree);
    fprintf(f, "%d\n", node->info.articles);
    fprintf(f, "%d\n", node->info.quotes);
    fprintf(f, "%d\n", node->info.hirshIndex);
    saveToFile(node->left, f);
}

```

```

        saveToFile(node->right, f);
    }

// Загрузка дерева из файла
struct tree *loadFromFile(FILE *f) {
    char buf[10];
    if (!fgets(buf, sizeof(buf), f)) return NULL;
    if (buf[0] == 'N') {
        return NULL;
    } else if (buf[0] == 'D') {
        struct scientist data;
        if (fscanf(f, "%d\n", &data.id) != 1) return NULL;
        if (fgets(data.name, sizeof(data.name), f) == NULL) return NULL;
        data.name[strcspn(data.name, "\n")] = '\0';
        if (fgets(data.area, sizeof(data.area), f) == NULL) return NULL;
        data.area[strcspn(data.area, "\n")] = '\0';
        if (fgets(data.degree, sizeof(data.degree), f) == NULL) return NULL;
        data.degree[strcspn(data.degree, "\n")] = '\0';
        if (fscanf(f, "%d\n", &data.articles) != 1) return NULL;
        if (fscanf(f, "%d\n", &data.quotes) != 1) return NULL;
        if (fscanf(f, "%d\n", &data.hirshIndex) != 1) return NULL;
        struct tree *node = (struct tree*)malloc(sizeof(struct tree));
        if (!node) return NULL;
        node->info = data;
        node->left = loadFromFile(f);
        node->right = loadFromFile(f);
        return node;
    }
    return NULL;
}

// Уровень самого правого узла
int findRightmostLevel(struct tree *node) {
    int level = 1;
    while (node->right) {
        node = node->right;
        level++;
    }
    return level;
}

// Вывод учёных с >100 статей (обход и подсчёт)
void printScientistsWithMoreThan100Articles(struct tree *node, int *count) {
    if (node) {
        printScientistsWithMoreThan100Articles(node->left, count);
        if (node->info.articles > 100) {
            printNode(node);
            (*count)++;
        }
        printScientistsWithMoreThan100Articles(node->right, count);
    }
}

// Освобождение памяти дерева
void freeTree(struct tree *node) {
    if (node) {
        freeTree(node->left);
        freeTree(node->right);
        free(node);
    }
}

```



```

printf("Введите научную область: ");
fgets(data.area, sizeof(data.area), stdin);
data.area[strcspn(data.area, "\n")] = '\0';

printf("Введите учёную степень: ");
fgets(data.degree, sizeof(data.degree), stdin);
data.degree[strcspn(data.degree, "\n")] = '\0';

printf("Введите количество научных статей: ");
scanf("%d", &data.articles);
printf("Введите количество цитирований: ");
scanf("%d", &data.quotes);
printf("Введите индекс Хирша: ");
scanf("%d", &data.hirshIndex);
clearInputBuffer();

printf("\n");
return data;
}

// Создание дерева через многократный ввод
struct tree *createTree() {
    struct tree *top = NULL;
    char ch;
    do {
        top = addNode(top, readData());
        system("cls");
        printf("\n\n1 - ввести ещё\n0 - закончить ввод\n-> ");
        ch = getch();
        printf("\n");
        if (ch != '1' && ch != '0') ch = '0';
    } while (ch == '1');
    return top;
}

// Главный цикл программы (меню, обработка команд)
void runProgram(struct tree **top) {
    int choice;
    char filename[256];
    FILE *f;
    int cnt, level, toDel;

    while (1) {
        system("cls");
        printf("===== МЕНЮ =====\n");
        printf("1. Организация дерева\n");
        printf("2. Просмотр дерева (обход)\n");
        printf("3. Отображение структуры дерева\n");
        printf("4. Добавить узел дерева\n");
        printf("5. Исключить узел из дерева\n");
        printf("6. Сохранить дерево в файл\n");
        printf("7. Загрузить дерево из файла\n");
        printf("8. Определить уровень самого правого узла\n");
        printf("9. Вывести учёных с более чем 100 статей\n");
        printf("ESC. Выход\n");
        printf("-> ");
        choice = getch();

        switch (choice) {
            case '1':
                system("cls");
                *top = createTree();
                break;
            case '2':

```

```

        system("cls");
        printTableHeader();
        viewTree(*top);
        printf("\nНажмите любую клавишу...");
        getch();
        break;
    case '3':
        system("cls");
        printf("\nСтруктура дерева (повёрнута влево):\n");
        printTreeStructure(*top, 0, 10);
        printf("\nНажмите любую клавишу...");
        getch();
        break;
    case '4':
        system("cls");
        *top = addNode(*top, readData());
        printf("Узел добавлен.\nНажмите любую клавишу...");
        getch();
        break;
    case '5':
        system("cls");
        printf("Введите количество статей удаляемого учёного: ");
        scanf("%d", &toDel);
        clearInputBuffer();
        *top = deleteNode(*top, toDel);
        printf("Нажмите любую клавишу...");
        getch();
        break;
    case '6':
        if (*top == NULL) {
            printf("Дерево пусто, сохранять нечего.\n");
            Sleep(1500);
            break;
        }
        printf("Введите имя файла для сохранения: ");
        fgets(filename, 256, stdin);
        filename[strcspn(filename, "\n")] = '\0';
        f = fopen(filename, "w");
        if (!f) {
            printf("Ошибка открытия файла для записи.\n");
            Sleep(1500);
            break;
        }
        saveToFile(*top, f);
        fclose(f);
        printf("Дерево сохранено в файл %s\n", filename);
        printf("Нажмите любую клавишу...");
        getch();
        break;
    case '7':
        system("cls");
        printf("Введите имя файла для загрузки: ");
        fgets(filename, 256, stdin);
        filename[strcspn(filename, "\n")] = '\0';
        f = fopen(filename, "r");
        if (!f) {
            printf("Ошибка открытия файла для чтения.\n");
            Sleep(1500);
            break;
        }
        struct tree *newnode = loadFromFile(f);
        fclose(f);
        if (newnode) {
            freeTree(*top);

```

```

        *top = newnode;
        printf("Дерево успешно загружено из файла %s\n", filename);
    } else {
        printf("Файл пуст или повреждён.\n");
    }
    printf("Нажмите любую клавишу...");
    getch();
    break;
case '8':
    system("cls");
    if (*top == NULL) {
        printf("Дерево пусто.\n");
        Sleep(1500);
        break;
    }
    level = findRightmostLevel(*top);
    printf("Самый правый узел находится на уровне: %d\n", level);
    printf("Нажмите любую клавишу...");
    getch();
    break;
case '9':
    system("cls");
    if (*top == NULL) {
        printf("Дерево пусто.\n");
        Sleep(1500);
        break;
    }
    cnt = 0;
    printf("\nУчёные с количеством статей > 100:\n");
    printTableHeader();
    printScientistsWithMoreThan100Articles(*top, &cnt);
    if (cnt == 0) {
        printf("Нет учёных с количеством статей более 100.\n");
    }
    printf("Нажмите любую клавишу...");
    getch();
    break;
case ESC:
    freeTree(*top);
    return;
default:
    printf("Команда не распознана!\n");
    Sleep(1500);
    break;
}
}
}

```

8. Тестирование и отладка программы

На рисунках 6.1 - 6.9 отображены результаты работы кода согласно всем условиям по заданию для языка C.

На рисунке 6.1 показан интерфейс меню.

На рисунке 6.2 показан процесс чтения дерева из текстового файла. Выбираем пункт меню 7, далее программа просит ввести название файла, после чего читает все данные из файла и выводит сообщение об успешно прочитанном дереве.

На рисунке 6.3 представлено содержание файла «to-test.txt».

На рисунке 6.4 представлено выполнение пункта меню 2 (просмотр дерева). В программе просмотр реализован обходом сверху-вниз. Данные представлены в виде таблицы для более лёгкого восприятия информации.

На рисунке 6.5 показан результат работы функции печати структуры дерева. Структура печатается с поворотом влево. Также каждая запись оформлена минимальным набором данных: ID, ФИО и кол-во научных статей.

На рисунке 6.6 показан результат поиска учёных с количеством статей > 100. Вывод стилизован в виде таблицы, для удобной работы с данными.

На рисунке 6.7 показан процесс добавления новой записи (узла) в дерево. Программа просит ввести все данные, затем сама автоматически ставит запись на нужное место в дереве, не нарушая его структуру.

На рисунке 6.8 показан процесс сохранения дерева в текстовый файл. В данном случае сохранение производится в файл «my-tree.txt».

На рисунке 6.9 показано содержимое файла «my-tree.txt» после сохранения дерева.

```
===== МЕНЮ =====  
1. Организация дерева  
2. Просмотр дерева (обход)  
3. Отображение структуры дерева  
4. Добавить узел дерева  
5. Исключить узел из дерева  
6. Сохранить дерево в файл  
7. Загрузить дерево из файла  
8. Определить уровень самого правого узла  
9. Вывести учёных с более чем 100 статей  
ESC. Выход  
-> |
```

Рисунок 6.1 – Интерфейс меню

```
Введите имя файла для загрузки: to-test.txt  
Дерево успешно загружено из файла to-test.txt  
Нажмите любую клавишу...|
```

Рисунок 6.2 – Процесс чтения дерева из текстового файла

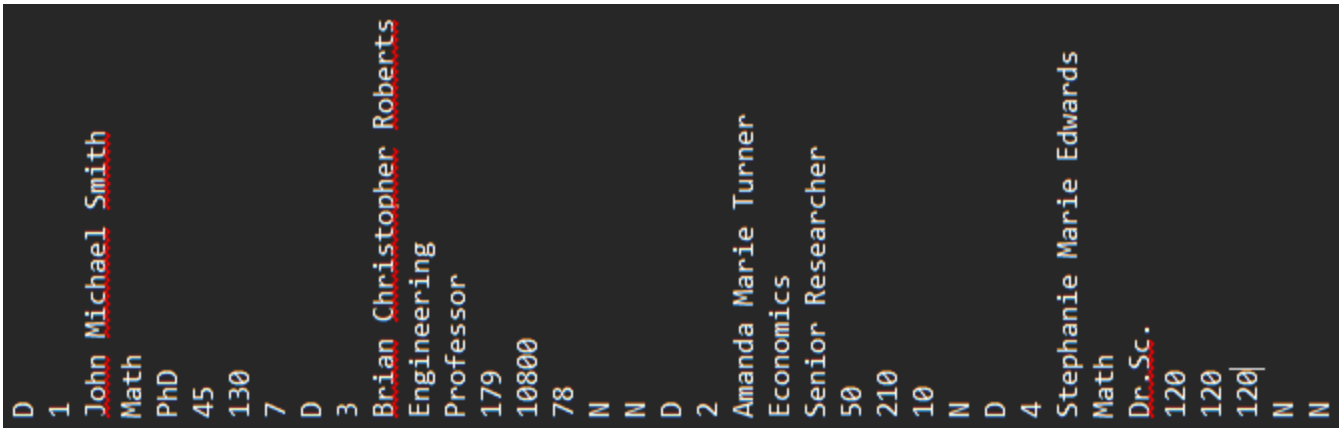


Рисунок 6.3 – Содержание файла to-test.txt

ID	Фамилия Имя Отчество	Учёная Степень	Область Науки	Кол-во статей	Кол-во цитирований	Индекс Хирша
3	Brian Christopher Roberts	Professor	Engineering	179	10800	78
1	John Michael Smith	PhD	Math	45	130	7
2	Amanda Marie Turner	Senior Researcher	Economics	50	210	10
4	Stephanie Marie Edwards	Dr.Sc.	Math	120	120	120

Нажмите любую клавишу...

Рисунок 6.4 – Результат обхода дерева сверху-вниз

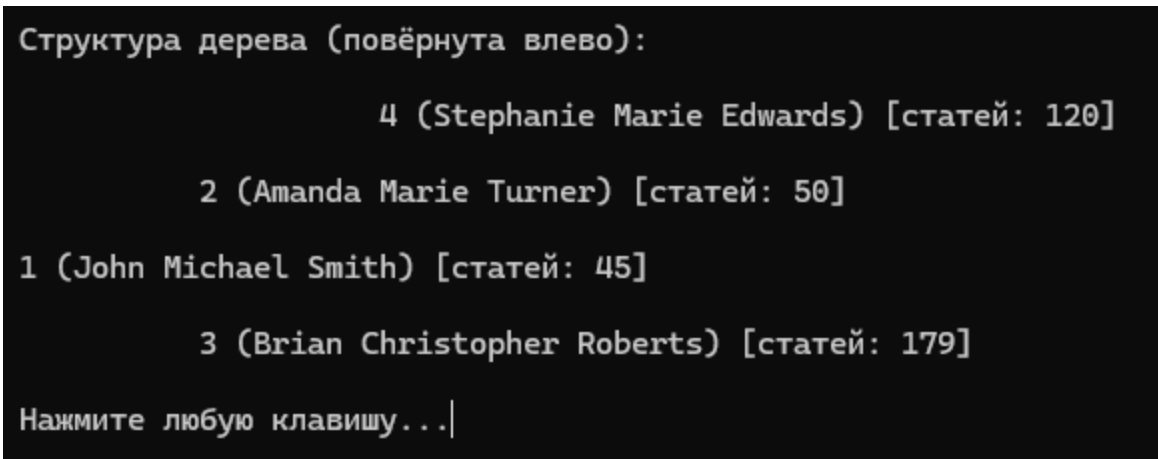


Рисунок 6.5 – Результат отображения структуры дерева

Учёные с количеством статей > 100:

3	Brian Christopher Roberts	Professor	Engineering	179	10800	78
4	Stephanie Marie Edwards	Dr.Sc.	Math	120	120	120

Нажмите любую клавишу...|

Рисунок 6.6 – Результат поиска учёных с количеством статей более 100

```

Введите ФИО: Kevin Michael Mitchell
Введите научную область: Chemistry
Введите учёную степень: Senior Researcher
Введите количество научных статей: 72
Введите количество цитирований: 2270
Введите индекс Хирша: 23

Узел добавлен.
Нажмите любую клавишу...|

```

Рисунок 6.7– Результат добавления нового узла в дерево

```

-> Введите имя файла для сохранения: my-tree.txt
Дерево сохранено в файл my-tree.txt
Нажмите любую клавишу...|

```

Рисунок 6.8– Процесс сохранения дерева в текстовый файл

```

D 1 John Michael Smith
  Math
  PhD
  45
  130
  7
  D
  3 Brian Christopher Roberts
    Engineering
    Professor
    179
    10800
    78
    N
    N
    D
    2 Amanda Marie Turner
      Economics
      Senior Researcher
      50
      210
      10
      N
      D
      4 Stephanie Marie Edwards
        Math
        Dr.Sc.
        120
        120
        120
        D
        2 Kevin Michael Mitchell
          Chemistry
          Senior Researcher
          72
          2270
          23
          N
          N
          N

```

Рисунок 6.9 – Содержимое текстового файла my-tree.txt

ВЫВОД

В ходе выполнения лабораторной работы была создана модульная программа на языке C, реализующая бинарное дерево как структуру данных. Код предыдущей лабораторной работы (№5) был разбит на логические компоненты: заголовочные файлы, содержащие описание структур и прототипы функций, и файлы с реализацией, отвечающие за работу с деревом и пользовательским интерфейсом. Такой модульный подход позволил упорядочить проект, облегчив его отладку и дальнейшее сопровождение.